



АУДИТ БЕЗОПАСНОСТИ СМАРТ-КОНТРАКТОВ

ICFT Protocol

Аудит безопасности кредитного протокола ICFT: учёт пула и индексная модель долга, оракульный слой, механизм ликвидаций и полномочия администратора.

0

● КРИТИЧЕСКАЯ

4

● ВЫСОКАЯ

5

● СРЕДНЯЯ

7

● НИЗКАЯ

3

● ИНФОРМАЦИОННАЯ

Сведения о документе

Заказчик	ICFT Protocol
Протокол	ICFT Protocol
Тип отчёта	Аудит безопасности смарт-контрактов
Гриф	Публичная версия
Аудитор	WhyUSayMyName
Контакт	github.com/WhyUSayMyName

История версий

ВЕРСИЯ	ДАТА	ОПИСАНИЕ
v1.0	12 сент. 2026	Первичный отчёт передан заказчику. Проверка исправлений не проводилась.

Об аудиторе

WhyUSayMyName, независимый blockchain security researcher. Работаю с кредитными рынками, хранилищами, оракульными слоями, DEX, стейблкоинами, governance, cross-chain взаимодействием и мостами.

В работе сочетаю построчный ручной разбор, инвариантное тестирование с состоянием и моделирование экономических сценариев. Прежде чем искать уязвимости, строю карту протокола: как контракты вызывают друг друга, какие функции меняют состояние, где хранится и куда движется стоимость, какие роли на что влияют и какие инварианты обязаны выполняться всегда. Карта показывает границы доверия и те узлы, где ошибка стоит дороже всего, и дальше основное внимание уходит именно туда.

Каждая находка должна быть подтверждена исполняемым доказательством, а не только рассуждением.

Использование отчёта

Отчёт описывает состояние кода на коммите, указанном в разделе «Объём аудита». Он передан заказчику для повышения безопасности проверенной системы и не является одобрением проекта, команды, токена или бизнес-модели. Аудит ограничен по времени и не гарантирует отсутствия уязвимостей. Полный текст ограничения ответственности приведён в Приложении С.

Отчёт публикуется с согласия заказчика. На момент публикации протокол не развёрнут в основной сети, а перечисленные находки команда закрывает до деплоя. Статус `открыто` у находок отражает состояние на дату аудита, а не наличие уязвимостей в будущем релизе.

Содержание

01	Резюме	05
02	Объём аудита	08
03	Методология	10
04	Обзор системы	12
05	Модель доверия и привилегированные роли	14
06	Классификация рисков	16
07	Зрелость кодовой базы	17
08	Сводка находок	18
09	Описание находок	20
● Н-01	Цена ICFT задаётся одной ролью мгновенно и без ограничений, что позволяет вывести стоимость из фонда А	20
● Н-02	Непокрытый долг остаётся в системе после изъятия всего обеспечения, механизма списания нет	22
● Н-03	Допустимое валидацией значение бонуса ликвидатора полностью отключает ликвидации	24
● Н-04	Реализация любого контракта подменяется одним ключом в том же блоке, без задержки	26
● М-01	Счётчик выданных токенов уменьшается на возвращённые, из-за чего утилизация занижается при падении цены ICFT	28
● М-02	Во время аварийной паузы обеспечение заперто, а проценты продолжают начисляться	30
● М-03	Выручка протокола накапливается в контракте без какой-либо возможности её использовать	32
● М-04	Минимальный размер займа не проверяется, что позволяет создавать позиции, невыгодные для ликвидации	34
● М-05	Передача административной роли одношаговая, а пауза не ограничена по времени	35
● L-01	Ответ Chainlink проверяется не полностью: номер раунда игнорируется, запас по устареванию отсутствует	37
● L-02	Валидация параметров риска допускает нулевой запас между целевым LTV и порогом ликвидации	38

● L-03	Сборка проекта не воспроизводится: конфликт путей зависимостей и нестрогая версия компилятора	39
● L-04	Оценка обеспечения делает внешний вызов на каждый актив в цикле по всему списку	40
● L-05	В обновляемых контрактах не зарезервированы слоты хранилища	41
● L-06	Обеспечение засчитывается по запрошенной сумме, а не по фактически полученной	42
● L-07	Ступенчатая кривая ставки даёт скачок на границе участка и не ограничена сверху	43
● I-01	Нестрогая версия компилятора и неполная документация публичного интерфейса	44
● I-02	Внешний вайтпейпер описывает протокол иначе, чем спецификация MVP и реализация	45
● I-03	Проценты начисляются сложным способом, а режим начисления в спецификации не зафиксирован	47
A	Инварианты и тесты свойств	48
B	Разбор автоматического анализа	49
C	Ограничение ответственности	50

01

Резюме

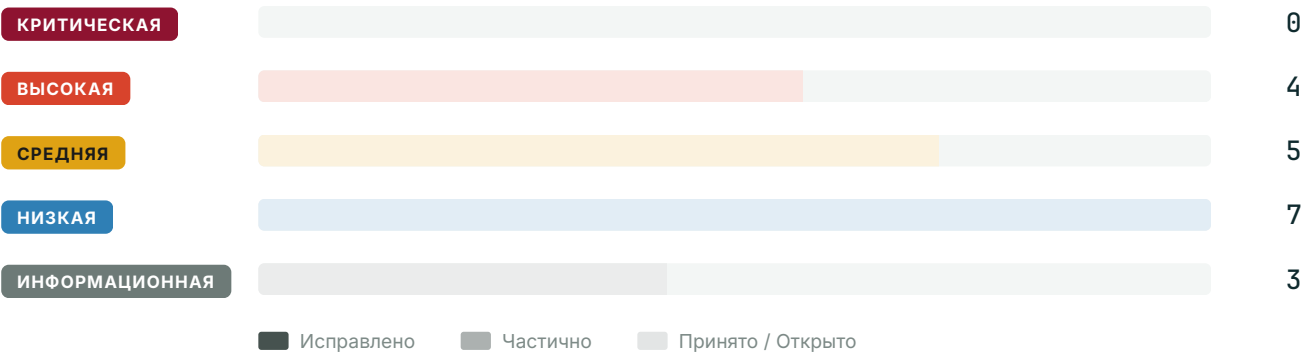
ICFT это кредитный протокол с одним рынком заимствования. Пользователь вносит ETH или токены обеспечения из белого списка, занимает ICFT и получает долг, номинированный в USD. Проценты начисляются через глобальный индекс заимствования, ликвидацией занимается отдельный контракт под ролью оператора, цены поступают из Chainlink для ETH и из ручного источника для ICFT.

Выявлено **девятнадцать проблем**: четыре высокой критичности, пять средней, семь низкой и три информационных. Критических проблем не выявлено, поскольку каждый путь к потере средств требует привилегированного ключа, а централизованная схема управления зафиксирована в спецификации как осознанное состояние MVP.

Основная часть находок сосредоточена вокруг двух тем: полномочия администратора, которые не ограничены ни задержкой, ни границами значений, и отсутствие модели неплатёжеспособности, из-за чего непокрытый долг остаётся в системе навсегда.

ТИП РАБОТ Приватный аудит, первичная проверка	ПЕРИОД с 1 по 12 сентября 2026	ТРУДОЗАТРАТЫ 12 дней · 1 аудитор
NSLOC В ОБЪЁМЕ 1 403	МЕТОДЫ Ручной анализ, статические анализаторы, инварианты Foundry, экономические сценарии	НАХОДКИ 19 · 0 исправлено

Обзор находок



ОБЩАЯ ОЦЕНКА

Ядро учёта сделано аккуратно. Долг хранится в масштабированных единицах и растёт через глобальный индекс, поэтому истёкшие периоды не переоцениваются задним числом более поздней ставкой. Направление округления в большинстве мест выбрано в пользу протокола, конвертация разрядностей между ETH, ICFT, USDT и ответом оракула выполнена симметрично и с явным указанием направления округления. Опасных конструкций в коде нет: ни `tx.origin`, ни `delegatecall`, ни `selfdestruct`, ни блоков `unchecked`.

Слабые места находятся на границах системы. Цена ICFT берётся из единственного источника, который задаётся вручную одной ролью без задержки, без ограничения шага и без сверки с рынком. Этого достаточно, чтобы за две транзакции вывести стоимость из общего фонда, что подтверждено исполняемым доказательством. Механизм ликвидаций закрывает здоровые сценарии корректно, но не описывает состояние, в котором долг превышает обеспечение: залог изымается полностью, остаток долга остаётся в системе, и списать его нечем.

Отдельно стоит отметить валидацию параметров риска. Она пропускает значения, которые выглядят разумно, но полностью отключают ликвидации из-за переполнения в расчёте. Это не кража средств, а отказ защитного механизма в тот момент, когда он нужнее всего.

Ключевые наблюдения

- 01 **Цена ICFT держится на доверии к одному ключу.** Источник цены единственный, второй для сверки отсутствует, ограничителя отклонения и границ значения нет. Роль, задающая цену, применяет её мгновенно. Спецификация допускает ручную цену для MVP, но не освобождает от ограничителей.
- 02 **Неплатёжеспособность не смоделирована.** При обвале обеспечения ликвидация забирает весь залог и закрывает только часть долга. Остаток продолжает существовать в учёте, механизма списания или страхового покрытия нет.
- 03 **Разумное значение параметра выключает ликвидации.** Бонус ликвидатора выше примерно 17,6 процента при целевом LTV 85 процентов принимается валидацией, но расчёт ликвидации после этого уходит в переполнение и откатывается. Ликвидации перестают работать для всего протокола.
- 04 **Учёт выданных токенов расходится с долгом при движении цены.** Счётчик заимствованных ICFT уменьшается на число возвращённых токенов, а не на изначально выданное. При падении цены ICFT погашение одного заёмщика обнуляет счётчик, хотя долг других остаётся, и утилизация занижается для всех.
- 05 **Тесты были, инвариантов на учёт фонда не было.** В репозитории есть модульные, фаззинг и инвариантные тесты, но ни один инвариант не проверял сохранение токенов фонда. Именно поэтому расхождение учёта не всплывало. Набор инвариантов из Приложения А закрывает этот пробел.

Приоритеты до деплоя

- 01 **Закрыть Н-01:** ввести задержку, ограничение шага и подпись нескольких лиц для функций, задающих цену и источник цены.
- 02 **Закрыть Н-02:** описать поведение при обеспечении ниже долга и завести учёт непокрытого долга.

- 03 Закреть **H-03**: ограничить бонус ликвидатора так, чтобы расчёт ликвидации не мог переполниться.
- 04 Включить инварианты из Приложения А в непрерывную интеграцию и прогонять их перед каждым релизом.

02

Объём аудита

Аудит охватывал контракты ядра протокола на указанном коммите. Проверка исправлений в этой версии отчёта не проводилась.

Репозиторий	ICFT-Protocol
Коммит	7135745
Язык	Solidity
Компилятор	0.8.35 (via-IR, optimizer 200 runs)
Фреймворк	Foundry

Файлы в объёме аудита

ФАЙЛ	NSLOC	SHA-256 (ПРЕФИКС)
src/core/ICFT/lending/LendingPool.sol	593	aa0630616182c9ce
src/core/ICFT/oracle/PriceOracle.sol	201	d7504f6aa940b932
src/core/ICFT/lending/LiquidationEngine.sol	200	b8699b6de11f408c
src/core/ICFT/risk/RiskEngine.sol	128	3d2a48ab876c7df5
src/core/ICFT/lending/InterestRateModel.sol	47	9b54fd492361405f
src/core/ICFT/token/ICFT.sol	45	3af482a4c65c488a
src/core/interfaces/ILendingPool.sol	62	0e4aa8660ddc8c55
src/core/interfaces/ILiquidationEngine.sol	36	96cfacbaa97fd355
src/core/interfaces/IRiskEngine.sol	23	78e011d289d14c0f
src/core/interfaces/IPriceOracle.sol	21	7e912ce42e02bb38
src/core/interfaces/IInterestRateModel.sol	16	970550165e83faea
src/core/utils/Errors.sol	26	bf3758c52264f8bd
src/core/utils/PriceSource.sol	5	a293a25df7081647
Итого	1 403	

Вне объёма

■ Скрипты деплоя, тесты и моки (`script/` , `test/` , `src/mocks/`).

- Сторонние зависимости: OpenZeppelin 5.x, интерфейсы Chainlink.
- Офчейн-инфраструктура бота ликвидаций, мониторинг и фронтенд.
- Собственный DEX, маршрутизация обмена и механика управления: вынесены за рамки MVP самой спецификацией.
- Вестинг, резервный фонд В и минимальный размер займа: в спецификации MVP отсутствуют.

03

Методология

Работа шла последовательно. Модель угроз, собранная в начале, определила, куда направить ручной разбор и фаззинг. Каждый этап опирался на результат предыдущего.

Этапы работ

01

Разведка и модель

Разбор документации, карта зависимостей контрактов, схема денежных потоков, перечень ролей и точек, работающих со средствами.

02

Статический анализ

Прогон Slither и solhint по всей кодовой базе, ручной разбор каждого срабатывания (Приложение В).

03

Ручной разбор по модулям

Построчная проверка токена, пула, модели ставки, оракула, ликвидаций, фондов, прав доступа и аварийной заморозки.

04

Инварианты и фаззинг

Обработчики Foundry с состоянием, двигающие в том числе цены; свойства перечислены в Приложении А.

05

Доказательства

Для каждой находки уровня Medium и выше написан исполняемый тест, воспроизводящий поведение с конкретными числами.

06

Экономические сценарии

Обвал обеспечения, рентабельность ликвидации с учётом газа, глубина пула ICFT, простой бота, давление продаж.

07

Валидация находок

Повторная враждебная проверка каждой находки против исходников и спецификации MVP. Часть находок скорректирована, часть переведена в разряд соответствующих спецификации.

Инструменты

ИНСТРУМЕНТ	НАЗНАЧЕНИЕ
Foundry	Доказательства, инвариантное тестирование с состоянием, дифференциальное сравнение с эталонной моделью.
Slither 0.11.5	Статический анализ, 101 детектор по всей кодовой базе.
solhint 6.2.4	Проверка стиля, документации и газовых практик.
Python	Эталонная модель формул LTV, процентов и частичной ликвидации; численные экономические сценарии.

Ограничения

Работа заняла двенадцать дней силами одного аудитора. Проверка исправлений в эту версию отчёта не входит. Экономические сценарии опираются на предположения о глубине пула ICFT, цене газа и распределении позиций: эти параметры вынесены в модель и меняются, реальные значения на момент запуска могут отличаться.

Поведение токенов обеспечения, отклоняющихся от стандарта ERC-20 (комиссия за перевод, ребейз, колбэки), оценивалось только по коду, без интеграционных прогонов.

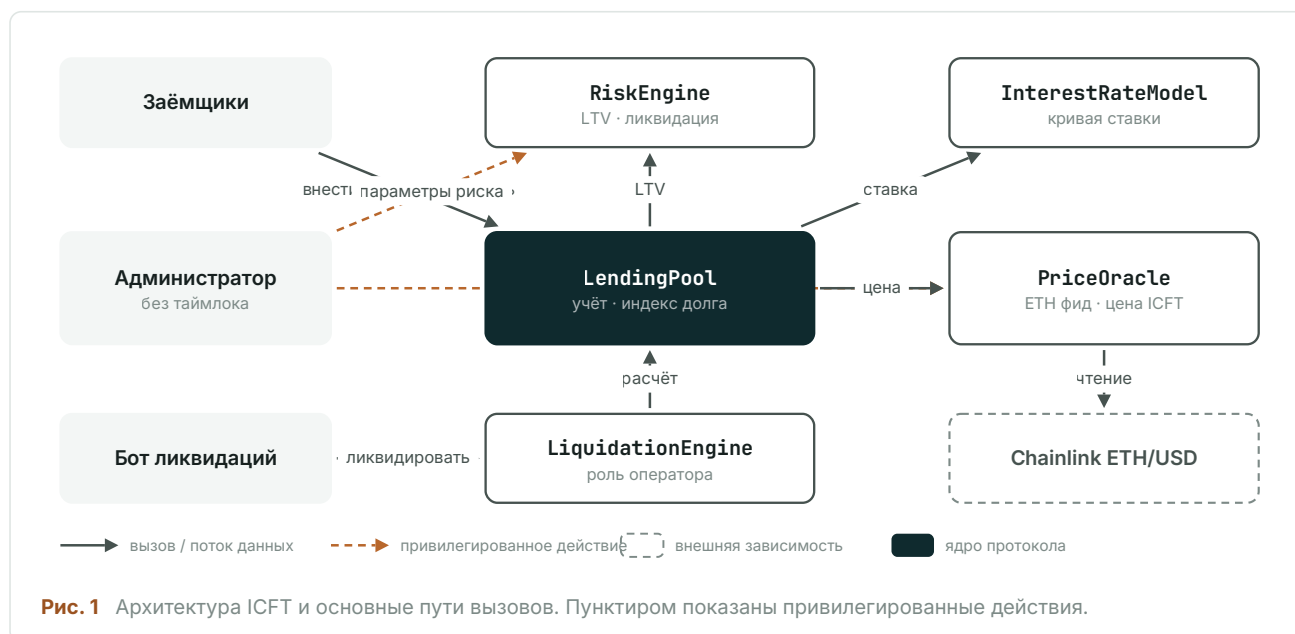
04

Обзор системы

ICFT это кредитный протокол с одним рынком заимствования. Пользователь вносит ETH или токен обеспечения из белого списка и занимает токен ICFT. Долг фиксируется в USD на момент выдачи, поэтому число токенов, необходимое для погашения, меняется вместе с ценой ICFT. Такое поведение прямо описано в спецификации MVP.

Проценты начисляются глобальным индексом заимствования: при каждом действии, меняющем состояние, индекс увеличивается на ставку за истёкший период, а долг позиции получается умножением масштабированной величины на текущий индекс. Ставку возвращает `InterestRateModel` по ступенчатой кривой утилизации, границы риска задаёт `RiskEngine`, цены отдаёт `PriceOracle`.

Ликвидация вызывается не свободно: `LendingPool.liquidate` закрыт ролью бота, а `LiquidationEngine.executeLiquidation` ролью оператора. Спецификация MVP допускает такую схему явно. Все контракты развёрнуты за прозрачными прокси.



Компоненты

LendingPool	Держит обеспечение и ликвидность фонда A, ведёт масштабированный долг и индекс заимствования, проверяет LTV при займе и выводе, исполняет расчёт ликвидации.
ICFT	ERC-20 с фиксированной эмиссией в один миллиард. Весь объём выпускается однократно при инициализации, функции дополнительной эмиссии нет.
PriceOracle	Нормализует цены к 1e18. ETH берётся из Chainlink с проверкой устаревания, цена ICFT задаётся вручную либо читается из фида.
RiskEngine	Считает LTV, определяет право на ликвидацию и размер частичного закрытия до целевого LTV, хранит параметры риска.
InterestRateModel	Ступенчатая кривая из четырёх участков, возвращает годовую ставку по утилизации и предел утилизации для новых займов.
LiquidationEngine	Обёртка над ликвидацией пула с предварительным расчётом. Забирает ICFT у оператора, передаёт обеспечение получателю, возвращает неиспользованный остаток.

05

Модель доверия и привилегированные роли

Протокол опирается на набор привилегированных ролей. По умолчанию все они и право на апгрейд достаются одному адресу деплоера, если переменные окружения не заданы иначе. В таблице показано, что может каждая роль и к чему приведёт её компрометация.

роль	кому принадлежит	полномочия	при компрометации
Владелец прокси	Один адрес, по умолчанию деплоер	Подменяет реализацию любого из шести контрактов через <code>ProxyAdmin</code> .	Полный контроль над протоколом и средствами. Апгрейд выполняется в том же блоке, окна для реакции пользователей нет (H-04).
ORACLE_ADMIN	Один адрес, по умолчанию деплоер	Задаёт цену ICFT и источник цены, меняет адреса фидов и предельный возраст ответа.	Произвольная оценка обязательств. Доказанный путь вывода стоимости из фонда A (H-01).
RISK_ADMIN	Один адрес, по умолчанию деплоер	Задаёт максимальный LTV, порог ликвидации, целевой LTV и бонус ликвидатора.	Значение бонуса в допустимом диапазоне полностью отключает ликвидации (H-03).
RATE_ADMIN	Один адрес, по умолчанию деплоер	Задаёт кривую ставки без верхнего предела.	Произвольно высокая ставка по действующим займам.
CONFIG_ADMIN	Один адрес, по умолчанию деплоер	Управляет буфером ликвидности и списком залоговых активов.	Добавление актива с некорректным фидом искажает оценку обеспечения.
PAUSER	Один адрес, по умолчанию деплоер	Останавливает внесение, заём, вывод и ликвидацию.	Залог заперт, проценты продолжают начисляться, снятие паузы не ограничено по времени (M-02, M-05).
LIQUIDATION_BOT, OPERATOR	Бот команды	Единственные, кто может запустить ликвидацию.	При простое бота нездоровые позиции не закрываются, внешнего резерва ликвидаторов нет.

Допущения о доверии

- Ключи ролей находятся под контролем команды и не скомпрометированы. Ни мультиподписи, ни задержки в коде и в настройках по умолчанию нет.
- Ручная цена ICFT отражает реальный рынок. Ончейн это не проверяется: ни второго источника, ни ограничителя отклонения не предусмотрено.
- Бот ликвидаций работает непрерывно и располагает достаточным запасом ICFT, чтобы закрывать позиции в момент рыночного стресса.
- Активы обеспечения ведут себя как стандартные ERC-20: без комиссии за перевод, ребейза и колбэков.
- Спецификация MVP считается управляющим документом. Расхождения с внешним вайтпейпером разобраны в находке I-02.

06

Классификация рисков

Критичность выводится из сочетания влияния на систему и вероятности реализации. Статус отражает состояние работы над проблемой на момент выпуска отчёта.

		ВЕРОЯТНОСТЬ		
		ВЫСОКАЯ	СРЕДНЯЯ	НИЗКАЯ
ВЛИЯНИЕ	ВЫСОКОЕ	КРИТИЧЕСКАЯ	ВЫСОКАЯ	СРЕДНЯЯ
	СРЕДНЕЕ	ВЫСОКАЯ	СРЕДНЯЯ	НИЗКАЯ
	НИЗКОЕ	СРЕДНЯЯ	НИЗКАЯ	НИЗКАЯ

Критичность определяется сочетанием влияния и вероятности.

Уровни критичности

КРИТИЧЕСКАЯ	Прямая и вероятная потеря средств пользователей или протокола либо необратимая неплатёжеспособность. Исправляется до деплоя.
ВЫСОКАЯ	Потеря средств или отказ защитного механизма при реалистичных, хотя и ограниченных условиях. Следует исправить до деплоя.
СРЕДНЯЯ	Ограниченные потери, потери при маловероятных условиях или существенное отклонение от заявленного поведения.
НИЗКАЯ	Незначительное отклонение от спецификации или общепринятых практик с малым прямым влиянием.
ИНФОРМАЦИОННАЯ	Качество кода, читаемость и эшелонированная защита. Прямого влияния на безопасность нет.

Статусы устранения

ИСПРАВЛЕНО	Проблема устранена, исправление проверено.
ЧАСТИЧНО	Исправление снижает риск, но не устраняет его полностью.
ПРИНЯТО	Команда принимает риск и не планирует менять код.
ОТКРЫТО	На момент выпуска отчёта исправление не предоставлено.

07

Зрелость кодовой базы

Оценка описывает инженерные практики в целом, помимо отдельных находок, и показывает, какие области требуют внимания в первую очередь.

ОБЛАСТЬ	ОЦЕНКА	КОММЕНТАРИЙ
Арифметика	 Удовлетворительная	Единая нормализация к 1e18, явное указание направления округления, округление долга вверх в пользу протокола. Блоков <code>unchecked</code> нет.
Контроль доступа	Слабая	Роли разделены на уровне кода, но по умолчанию сходятся к одному адресу. Ни задержки, ни мультиподписи, передача прав одношаговая (H-04, M-05).
Управление сложностью	 Удовлетворительная	Контракты разделены по зонам ответственности, расчёт ликвидации вынесен в отдельный модуль.
Децентрализация	 Отсутствует	Все полномочия и право апгрейда у одного ключа, апгрейд выполняется мгновенно. Спецификация относит финальную схему прокси за рамки MVP.
Документация	 Умеренная	Спецификация MVP подробная и совпадает с кодом по ключевым величинам. В коде 250 замечаний к NatSpec, часть публичных функций не описана (I-01).
События и мониторинг	 Умеренная	События есть на всех операциях со средствами, но ключевые поля не индексируются, что усложняет офлайн-наблюдение.
Обработка неплатёжеспособности	 Отсутствует	Состояние, в котором долг превышает обеспечение, не смоделировано. Непокрытый остаток остаётся в учёте без механизма списания (H-02).
Низкоуровневый код	Сильная	Ни <code>tx.origin</code> , ни <code>delegatecall</code> , ни <code>selfdestruct</code> в объёме аудита. Два низкоуровневых вызова служат отправке ETH и проверяют результат.
Тестирование	 Умеренная	Есть модульные, фаззинг и инвариантные тесты, но сохранение токенов фонда ни одним инвариантом не проверялось, поэтому расхождение учёта не обнаруживалось (M-01).

08

Сводка находок

ID	НАЗВАНИЕ	КРИТИЧНОСТЬ	СТАТУС
H-01	Цена ICFT задаётся одной ролью мгновенно и без ограничений, что позволяет вывести стоимость из фонда A	ВЫСОКАЯ	ОТКРЫТО
H-02	Непокрытый долг остаётся в системе после изъятия всего обеспечения, механизма списания нет	ВЫСОКАЯ	ОТКРЫТО
H-03	Допустимое валидацией значение бонуса ликвидатора полностью отключает ликвидации	ВЫСОКАЯ	ОТКРЫТО
H-04	Реализация любого контракта подменяется одним ключом в том же блоке, без задержки	ВЫСОКАЯ	ОТКРЫТО
M-01	Счётчик выданных токенов уменьшается на возвращённые, из-за чего утилизация занижается при падении цены ICFT	СРЕДНЯЯ	ОТКРЫТО
M-02	Во время аварийной паузы обеспечение заперто, а проценты продолжают начисляться	СРЕДНЯЯ	ОТКРЫТО
M-03	Выручка протокола накапливается в контракте без какой-либо возможности её использовать	СРЕДНЯЯ	ОТКРЫТО
M-04	Минимальный размер займа не проверяется, что позволяет создавать позиции, невыгодные для ликвидации	СРЕДНЯЯ	ОТКРЫТО
M-05	Передача административной роли одношаговая, а пауза не ограничена по времени	СРЕДНЯЯ	ОТКРЫТО
L-01	Ответ Chainlink проверяется не полностью: номер раунда игнорируется, запас по устареванию отсутствует	НИЗКАЯ	ОТКРЫТО
L-02	Валидация параметров риска допускает нулевой запас между целевым LTV и порогом ликвидации	НИЗКАЯ	ОТКРЫТО
L-03	Сборка проекта не воспроизводится: конфликт путей зависимостей и нестрогая версия компилятора	НИЗКАЯ	ОТКРЫТО
L-04	Оценка обеспечения делает внешний вызов на каждый актив в цикле по всему списку	НИЗКАЯ	ОТКРЫТО
L-05	В обновляемых контрактах не зарезервированы слоты хранилища	НИЗКАЯ	ОТКРЫТО
L-06	Обеспечение засчитывается по запрошенной сумме, а не по фактически полученной	НИЗКАЯ	ОТКРЫТО
L-07	Ступенчатая кривая ставки даёт скачок на границе участка и не ограничена сверху	НИЗКАЯ	ОТКРЫТО

ID	НАЗВАНИЕ	КРИТИЧНОСТЬ	СТАТУС
I-01	Нестрогая версия компилятора и неполная документация публичного интерфейса	ИНФОРМАЦИОННАЯ	ОТКРЫТО
I-02	Внешний вайтпейпер описывает протокол иначе, чем спецификация MVP и реализация	ИНФОРМАЦИОННАЯ	ОТКРЫТО
I-03	Проценты начисляются сложным способом, а режим начисления в спецификации не зафиксирован	ИНФОРМАЦИОННАЯ	ОТКРЫТО

09

Описание находок

Н-01

ВЫСОКАЯ

ОТКРЫТО

Цена ICFT задаётся одной ролью мгновенно и без ограничений, что позволяет вывести стоимость из фонда А

ВЛИЯНИЕ

Высокое

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Оракульный слой

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/oracle/PriceOracle.sol#L243-L245
src/core/ICFT/oracle/PriceOracle.sol#L288-L296
src/core/ICFT/oracle/PriceOracle.sol#L198-L204
```

ОПИСАНИЕ

Цена ICFT берётся из единственного источника. Поле `icftPriceSource` по умолчанию равно `Manual`, и тогда `getICFTUSDPrice()` возвращает значение, записанное администратором вручную. Проверок в `_setManualICFTPrice()` всего две: цена не равна нулю и разрядность не больше восемнадцати.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
288 function _setManualICFTPrice(uint256 price, uint8 decimals_) internal {
289     if (price == 0) revert InvalidManualPrice();
290     if (decimals_ > 18) revert UnsupportedPriceDecimals();
291
292     manualICFTPrice = price;
293     manualICFTPriceDecimals = decimals_;
```

Ни задержки между заявкой и применением, ни ограничения шага изменения, ни сверки с рыночным источником, ни границ допустимого значения нет. Функции `setManualICFTPrice`, `setICFTPriceSource`, `setNativeUSDFeed`, `setICFTUSDFeed`, `setCollateralAssetFeed` и `setMaxPriceAge` закрыты одной ролью `ORACLE_ADMIN_ROLE` и применяются в том же блоке.

Спецификация MVP допускает ручную цену как временное решение, но из этого не следует, что цену можно менять без ограничений.

ВЛИЯНИЕ

Долг номинирован в USD, поэтому цена ICFT определяет и то, сколько токенов выдаётся под заданный долг, и то, сколько токенов нужно вернуть для его закрытия. Отсюда два симметричных способа извлечь стоимость из общей ликвидности.

Если занижить цену перед выдачей займа, заёмщик получит больше токенов на тот же USD-долг. После возврата цены к рыночной он закроет долг меньшим числом токенов, а разница останется у него. Если зависеть цену перед погашением, действующий долг закроется меньшим числом токенов, и обеспечение выводится целиком.

Обе операции исполняются одним держателем роли и не требуют согласия других участников. Источник средств в обоих случаях это ликвидность фонда A, то есть средства протокола.

ДОКАЗАТЕЛЬСТВО (РОС)

Оба сценария воспроизведены на локальном развёртывании протокола.

Занижение цены до `$0.50` в момент выдачи: под то же обеспечение выдано `3 200 ICFT` вместо `1 600 ICFT` при цене `$1`. Долг зафиксирован в размере `$1 600`. После возврата цены к `$1` для полного погашения требуется `1 600 ICFT`, то есть у заёмщика остаётся `1 600 ICFT` чистой прибыли.

Завышение цены до `$2` в момент погашения: долг, взятый как `1 600 ICFT`, закрывается передачей `800 ICFT`, после чего обеспечение выводится полностью.

```
1 forge test --mt test_S3_manipulateDownAtBorrow_freeTokens
2 forge test --mt test_S4_manipulateUpAtRepay_cheapClose
```

РЕКОМЕНДАЦИЯ

Ограничить любое изменение цены и источника цены так, чтобы одна скомпрометированная роль не могла переоценить обязательства мгновенно.

Ввести задержку между заявкой на новую цену и её применением, достаточную для того, чтобы заёмщики успели среагировать. Ограничить шаг изменения относительно действующего значения, чтобы отклонение за одну операцию оставалось в разумных пределах. Перевести роль под подпись нескольких лиц.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
function _setManualICFTPrice(uint256 price, uint8 decimals_) internal {
    if (price == 0) revert InvalidManualPrice();
    if (decimals_ > 18) revert UnsupportedPriceDecimals();
+   uint256 next = normalizePrice(price, decimals_);
+   uint256 prev = manualICFTPrice == 0 ? next : normalizePrice(manualICFTPrice,
    manualICFTPriceDecimals);
+   uint256 delta = next > prev ? next - prev : prev - next;
+   if (delta * BPS > prev * maxPriceDeviationBps) revert PriceDeviationTooLarge();

    manualICFTPrice = price;
    manualICFTPriceDecimals = decimals_;
```

Отдельно стоит завести второй источник для сверки и ограничитель, который останавливает работу протокола при расхождении источников выше порога. До запуска в основной сети это условие следует считать блокирующим: пока цена держится на доверии к одному ключу, все обязательства протокола держатся на нём же.

H-02

ВЫСОКАЯ

ОТКРЫТО

Непокрытый долг остаётся в системе после изъятия всего обеспечения, механизма списания нет

ВЛИЯНИЕ

Высокое

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Логика ликвидаций

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/lending/LendingPool.sol#L610-L659  
src/core/ICFT/lending/LendingPool.sol#L577-L608
```

ОПИСАНИЕ

Расчёт ликвидации в `_calculateLiquidationSettlement()` ограничивает изъятие остатком обеспечения заёмщика и уменьшает покрываемый долг до той величины, которую это обеспечение оплачивает с учётом бонуса.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
635 if (seizeAmount > collateralBalance) {  
636     seizeAmount = collateralBalance;  
637 }  
638  
639 uint256 actualSeizedValueUSD = riskEngine.getCollateralValueUSD(collateralAsset, seizeAmount);
```

Такое поведение корректно в том смысле, что вызов не откатывается и ликвидатор получает ровно то, что есть. Проблема в другом: когда обеспечения не хватает на весь долг, разница никуда не девается. Позиция остаётся с положительным `scaledDebtUSD` и нулевым обеспечением, а в протоколе нет ни счётчика непокрытого долга, ни функции списания, ни резерва, из которого эту разницу можно было бы покрыть.

Формально позиция продолжает считаться подлежащей ликвидации, потому что `calculateLTV` при нулевом обеспечении возвращает максимальное значение. Но изымать уже нечего, и каждый следующий вызов ликвидации отклоняется.

ВЛИЯНИЕ

Остаток долга навсегда остаётся в совокупных величинах `totalScaledDebtUSD` и `totalPrincipalDebtUSD`. Он продолжает расти вместе с индексом заимствования, хотя за ним не стоит ни обеспечения, ни платёжеспособного заёмщика. Совокупный долг протокола оказывается завышен, а выданные под этот долг токены ICFT в фонд А не вернутся.

Чем резче движение цены обеспечения, тем больше таких позиций. Численная модель на книге из тысячи позиций со смещением к предельному LTV показывает, что падение ETH на сорок процентов делает подлежащими ликвидации около девяноста четырёх процентов позиций, из которых примерно восемьдесят девять процентов уходят ниже стоимости обеспечения. Это уже не единичный случай, а состояние, в которое протокол попадает при обычном для рынка стрессе.

Отдельно усугубляет картину то, что ликвидация закрыта ролью и требует от ликвидатора запаса ICFT. Внешние участники подставиться не могут, а простой бота на сутки при падении обеспечения на четверть в той же модели даёт около восьмисот незакрытых позиций.

ДОКАЗАТЕЛЬСТВО (РОС)

Позиция с обеспечением в один ETH и долгом около \$1 600 после падения ETH до \$1 000 имеет LTV 160%. Ликвидация изымает всё обеспечение до нуля и закрывает только часть долга.

Остаточный долг после ликвидации составляет \$647.62 при нулевом обеспечении. Дальнейшие попытки ликвидации отклоняются, средств для покрытия остатка в протоколе нет.

```
1 forge test --mt test_P3_05_badDebtSurvivesFullLiquidation
```

РЕКОМЕНДАЦИЯ

Описать состояние неплатёжеспособности явно, а не оставлять его следствием арифметики.

Завести отдельный учёт непокрытого долга. Когда изъятие ограничено остатком обеспечения и часть долга остаётся непокрытой, списывать этот остаток с позиции и переносить его в накопитель безнадежного долга, уменьшая совокупные величины. Тогда позиция закрывается окончательно, а совокупный долг перестаёт содержать заведомо невозвратную часть.

Далее нужно решить, за чей счёт покрывается накопитель. Варианты обычные: резерв из выручки протокола, страховой фонд или распределение убытка между поставщиками ликвидности. Выбор относится к экономике протокола, но он должен быть сделан и зафиксирован в коде, иначе убыток остаётся неучтённым.

Дополнительно стоит рассмотреть снятие ролевого ограничения с ликвидации или запуск резервных ликвидаторов, чтобы позиции закрывались до перехода в неплатёжеспособность, а не после.

H-03

ВЫСОКАЯ

ОТКРЫТО

Допустимое валидацией значение бонуса ликвидатора полностью отключает ликвидации

ВЛИЯНИЕ

Высокое

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Параметры риска

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/risk/RiskEngine.sol#L233-L253
src/core/ICFT/risk/RiskEngine.sol#L180-L184
```

ОПИСАНИЕ

Валидация параметров риска проверяет бонус ликвидатора только по верхней границе в сто процентов.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
239 bool invalid = maxLtvBps_ == 0
240     || maxLtvBps_ >= liquidationThresholdBps_
241     || targetLtvBps_ > liquidationThresholdBps_
242     || liquidationThresholdBps_ > BPS
243     || liquidationBonusBps_ > BPS;
```

Расчёт ликвидации при этом использует бонус в знаменателе, который вычисляется как разность без защиты от перехода через ноль.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
180 uint256 denominator = (BPS * BPS) - (targetLtvBps * (BPS + liquidationBonusBps));
181 if (denominator == 0) revert InvalidRiskParameters();
```

Проверка на ноль есть, а на отрицательный результат нет, потому что в арифметике без знака его не существует: выражение просто переполняется и вызов откатывается. Условие работоспособности выглядит так: произведение целевого LTV на сумму базы и бонуса должно оставаться меньше квадрата базы. При целевом LTV в восемьдесят пять процентов это даёт предел бонуса около семнадцати и шести десятых процента. Валидация о нём не знает.

ВЛИЯНИЕ

Бонус в двадцать процентов не выглядит подозрительно и вполне может быть выставлен при настройке параметров под рыночные условия. Транзакция пройдёт успешно, и никакого сигнала об ошибке не будет. После этого любой вызов `calculateLiquidation` завершается откатом, а вместе с ним перестают работать `liquidate`, `executeLiquidation` и предварительный расчёт в движке ликвидаций.

Протокол теряет единственный механизм, который защищает его от накопления безнадёжного долга, и теряет его целиком, а не для отдельной позиции. Обнаружится это не в момент изменения параметра, а при первой попытке ликвидации, то есть уже во время рыночного стресса. Восстановить работу можно только повторным изменением параметров, что требует доступа к роли и времени.

Ошибка обратима, средства напрямую не теряются, поэтому это не критический уровень. Но отказ защитного механизма именно тогда, когда он нужен, влечёт те же последствия, что и H-02.

ДОКАЗАТЕЛЬСТВО (РОС)

Значение бонуса в двадцать процентов принимается функцией `setRiskParameters` без ошибок, после чего `calculateLiquidation` откатывается. То же верно для ста процентов. При семнадцати процентах расчёт ещё выполняется, что подтверждает границу около семнадцати и шести десятых процента.

```
1 forge test --mt test_P3_AC3_highBonusBricksLiquidation
```

РЕКОМЕНДАЦИЯ

Проверять не абстрактную границу в сто процентов, а само условие, при котором расчёт остаётся корректным. Это единственная проверка, которая действительно защищает от ошибки настройки.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
bool invalid = maxLtvBps_ == 0
    || maxLtvBps_ >= liquidationThresholdBps_
    || targetLtvBps_ > liquidationThresholdBps_
    || liquidationThresholdBps_ > BPS
    || liquidationBonusBps_ > BPS;

+
+ // знаменатель расчёта ликвидации должен оставаться положительным
+ if (targetLtvBps_ * (BPS + liquidationBonusBps_) >= BPS * BPS) {
+     revert InvalidRiskParameters();
+ }
```

Полезно также задать самостоятельный разумный потолок бонуса, порядка пятнадцати процентов, чтобы отсеять значения, экономически бессмысленные даже при корректной арифметике.

H-04

ВЫСОКАЯ

ОТКРЫТО

Реализация любого контракта подменяется одним ключом в том же блоке, без задержки

ВЛИЯНИЕ

Высокое

ВЕРОЯТНОСТЬ

Низкая

КАТЕГОРИЯ

Права доступа и апгрейд

РАСПОЛОЖЕНИЕ

```
script/DeployICFTProtocol.s.sol#L75-L80  
script/DeployICFTProtocol.s.sol#L162-L172
```

ОПИСАНИЕ

Все шесть контрактов протокола развёрнуты за `TransparentUpgradeableProxy`. Право на апгрейд принадлежит владельцу соответствующего `ProxyAdmin`, и вызов `upgradeAndCall` выполняется немедленно. Таймлока в схеме нет.

Владелец задаётся при развёртывании и по умолчанию совпадает с администратором протокола, который, в свою очередь, по умолчанию совпадает с деплоером.

```
script/DeployICFTProtocol.s.sol
```

solidity

```
163 config.admin = vm.envOr("ADMIN_ADDRESS", deployer);  
164 config.upgradeAdmin = vm.envOr("UPGRADE_ADMIN_ADDRESS", config.admin);
```

Тем же способом задаются получатели всех долей эмиссии. Если переменные окружения не заданы, один адрес получает все роли во всех модулях, право апгрейда и весь миллиард токенов.

ВЛИЯНИЕ

Апгрейд снимает все остальные ограничения, потому что заменяет саму логику. Новая реализация может добавить эмиссию в токен с якобы фиксированным объёмом, изменить учёт долга, вывести обеспечение или ликвидность фонда А. Никакие проверки, описанные в других находках, при этом не действуют.

Существенно здесь не то, что такое полномочие есть, а то, что оно выполняется мгновенно. У пользователей нет интервала, за который они могли бы заметить готовящееся изменение и закрыть позиции. Любая компрометация ключа сразу становится необратимой потерей средств.

Вероятность оценена как низкая, поскольку сценарий требует компрометации или недобросовестности держателя ключа, а не ошибки в коде. Влияние при этом максимальное, что и даёт высокий уровень.

Спецификация MVP относит финальную схему прокси за рамки текущего этапа, поэтому для тестовой сети это приемлемое состояние. Для основной сети его следует считать блокирующим.

ДОКАЗАТЕЛЬСТВО (POC)

Владелец `ProxyAdmin` подменяет реализацию `PriceOracle` на произвольную в рамках одной транзакции. Проверка подтверждает, что отметка времени не изменилась, то есть задержки между решением и его применением не существует.

```
1 forge test --mt test_P3_AC2_instantUpgradeNoTimeLock
```

РЕКОМЕНДАЦИЯ

Передать владение всеми экземплярами `ProxyAdmin` контракту `TimelockController` с задержкой, сопоставимой со временем, за которое пользователь может закрыть позицию. Обычная практика это интервал от суток до трёх.

Владельцем таймлока сделать кошелёк с подписью нескольких лиц, а не один адрес. Отдельно стоит разделить роль администратора протокола и роль владельца апгрейда: сейчас они по умолчанию совпадают, поэтому компрометация одного ключа даёт и оперативное управление, и возможность подменить код.

Перед развёртыванием в основной сети имеет смысл добавить в скрипт проверку, которая откажется продолжать, если адреса ролей и получателей эмиссии не заданы явно. Текущее поведение с подстановкой деплоера по умолчанию удобно для локальных прогонов, но опасно тем, что молча приводит к полной централизации.

M-01

СРЕДНЯЯ

ОТКРЫТО

Счётчик выданных токенов уменьшается на возвращённые, из-за чего утилизация занижается при падении цены ICFT

ВЛИЯНИЕ

Среднее

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Учёт пула

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/lending/LendingPool.sol#L341-L343
src/core/ICFT/lending/LendingPool.sol#L786-L788
src/core/ICFT/lending/LendingPool.sol#L848-L850
```

ОПИСАНИЕ

Утилизация считается как отношение выданных токенов ICFT к объёму фонда A. Такая метрика прямо предписана спецификацией MVP, вопрос не к ней, а к тому, как ведётся числитель.

При выдаче займа `totalBorrowedICFT` увеличивается на выданное число токенов. При погашении он уменьшается на число токенов, которые вернул заёмщик, причём с насыщением снизу.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
341 fundALiquidityICFT += returnedPrincipalICFT;
342 protocolRevenueICFT += returnedRevenueICFT;
343 totalBorrowedICFT = _saturatingSub(totalBorrowedICFT, returnedPrincipalICFT);
```

Долг номинирован в USD, поэтому число возвращённых токенов не обязано совпадать с числом выданных. При падении цены ICFT заёмщик возвращает больше токенов, чем брал, и вычитается из общего счётчика тоже больше. Разница списывается не с его займа, а с общего счётчика, то есть за счёт остальных заёмщиков. Насыщение в `_saturatingSub` скрывает переполнение и доводит счётчик до нуля.

ВЛИЯНИЕ

Утилизация перестаёт отражать реальную загрузку фонда. Поскольку от неё зависит ставка, заниженная утилизация переводит кривую в нижний участок, и все действующие заёмщики платят меньше положенного. При обратном движении цены эффект противоположный: счётчик остаётся выше нуля после полного погашения, и ставка оказывается завышенной.

Прямой потери средств здесь нет, речь об экономической точности и о корректности учёта. Существенно, что искажение накапливается при каждом погашении на изменившейся цене и не имеет механизма самокоррекции.

Сюда же относится инвариант `INV-08`: сумма ликвидности фонда и выданных токенов перестаёт равняться выделенному объёму фонда.

ДОКАЗАТЕЛЬСТВО (РОС)

Два заёмщика берут по `1 600 ICFT`, общий счётчик равен `3 200 ICFT`. Цена ICFT падает в десять раз, первый заёмщик закрывает свой долг, для чего возвращает `16 000 ICFT`.

После этого `totalBorrowedICFT` равен нулю, хотя второй заёмщик всё ещё должен `$1 600`. Утилизация показывает ноль при непогашенном долге.

```
1 forge test --mt test_F2_07_utilizationDistortedWithTwoBorrowers
```

РЕКОМЕНДАЦИЯ

Уменьшать счётчик не на фактически возвращённое число токенов, а на ту часть, которая соответствует закрытой доле долга этого заёмщика. Простой вариант это хранить в позиции число изначально выданных токенов и вычитать из общего счётчика пропорционально погашенной доле основного долга, а не по факту полученных токенов.

Насыщение в `_saturatingSub` на этом участке стоит убрать или заменить на явную ошибку. Сейчас оно маскирует ситуацию, которая при корректном учёте возникать не должна, и тем самым мешает обнаружить расхождение.

Инвариант `INV-08` из Приложения А следует включить в непрерывную интеграцию: именно его отсутствие в существующем наборе тестов оставляло это расхождение незамеченным.

M-02

СРЕДНЯЯ

ОТКРЫТО

Во время аварийной паузы обеспечение заперто, а проценты продолжают начисляться

ВЛИЯНИЕ

Среднее

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Аварийная заморозка

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/lending/LendingPool.sol#L268-L275  
src/core/ICFT/lending/LendingPool.sol#L420-L422
```

ОПИСАНИЕ

Пауза закрывает внесение, заём, вывод обеспечения и ликвидацию. Погашение остаётся доступным, и это осознанное решение, зафиксированное в спецификации MVP.

Начисление процентов паузой не закрыто. Функция `accrueInterest()` публичная и не имеет модификатора `whenNotPaused`, а `repay()` в любом случае синхронизирует позицию и двигает глобальный индекс.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
420 function accrueInterest() external {  
421     _accrueGlobalInterest();  
422 }
```

Получается сочетание, при котором долг продолжает расти, а вывести обеспечение нельзя. Заёмщик может только вносить средства в систему, но не забирать их.

ВЛИЯНИЕ

Заёмщик, который полностью погасил долг во время паузы, всё равно не может забрать обеспечение до её снятия. Заёмщик, который погасить не может, наблюдает рост долга без возможности выйти. Ликвидация в это время тоже отключена, поэтому позиции, ушедшие ниже порога, остаются открытыми и деградируют, а после снятия паузы часть из них окажется уже неплатёжеспособной, что усиливает H-02.

Отдельно отмечу, что спецификация не оговаривает поведение начисления во время паузы. То есть это не нарушение заявленного поведения, а незакрытый вопрос, который в сочетании с запретом на вывод работает против пользователя.

ДОКАЗАТЕЛЬСТВО (РОС)

Позиция с обеспечением в пять ETH и долгом `$5 000.00` на момент постановки на паузу. Через тридцать дней паузы долг составляет `$5 020.55`. Попытка вывести обеспечение откатывается, погашение при этом проходит.

```
1 forge test --mt test_P3_AC4_freezeLocksCollateralWhileInterestAccrues
```

РЕКОМЕНДАЦИЯ

Определиться с назначением паузы и привести поведение в соответствие.

Если пауза защищает протокол от продолжающегося ущерба, разумно остановить и начисление процентов на время заморозки, чтобы пользователь не платил за период, в который он лишён возможности управлять позицией. Технически достаточно не двигать `lastAccrualTime` вперёд на интервал паузы.

Если начисление решено оставить, следует разрешить вывод обеспечения тем позициям, у которых нет долга: такой вывод не создаёт риска для протокола, а пользователя из ловушки выпускает. И в том, и в другом случае выбранное поведение нужно описать в документации, чтобы оно не оставалось неявным следствием набора модификаторов.

Стоит также пересмотреть блокировку ликвидации во время паузы: сейчас заморозка останавливает единственный механизм, который сокращает плохой долг.

M-03

СРЕДНЯЯ

ОТКРЫТО

Выручка протокола накапливается в контракте без какой-либо возможности её использовать

ВЛИЯНИЕ

Среднее

ВЕРОЯТНОСТЬ

Высокая

КАТЕГОРИЯ

Учёт пула

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/lending/LendingPool.sol#L172
src/core/ICFT/lending/LendingPool.sol#L341-L342
src/core/ICFT/lending/LendingPool.sol#L502-L505
```

ОПИСАНИЕ

Процентная часть каждого погашения отделяется от основного долга и записывается в `protocolRevenueICFT`. Оттуда же она вычитается из объёма, доступного для новых займов.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
502 function getSpendablePrincipalBalance() public view returns (uint256 spendablePrincipalICFT) {
503     uint256 rawBalance = icft.balanceOf(address(this));
504     return _saturatingSub(rawBalance, protocolRevenueICFT);
505 }
```

Переменная встречается в контракте пять раз: объявление, два увеличения при погашении и ликвидации и два чтения. Функции, которая бы её уменьшала, выводила накопленное или распределяла между участниками, в контракте нет.

ВЛИЯНИЕ

Токены ICFT, соответствующие накопленной выручке, остаются на балансе контракта и при этом исключены из доступной ликвидности. Их нельзя ни выдать в заём, ни забрать. Фактически они изъяты из обращения без явного намерения их сжечь.

Объём растёт непрерывно вместе с погашениями. Ни поставщики ликвидности, ни казна протокола получить эту выручку не могут, то есть экономическая модель, в которой проценты кому-то достаются, в коде не реализована. Единственный способ добраться до этих средств сегодня это апгрейд реализации, что возвращает нас к H-04.

Вероятность высокая, потому что это не сценарий атаки, а штатный ход работы протокола: достаточно первого же погашения с процентами.

ДОКАЗАТЕЛЬСТВО (POC)

Заём на десять тысяч ICFT, год начисления, полное погашение. После этого `protocolRevenueICFT` равен `500 ICFT`, и ровно на эту величину уменьшен объём, доступный для новых займов. Поиск по исходникам подтверждает, что ни одна функция не уменьшает эту переменную.

```
1 forge test --mt test_P3_FUND2_revenueAccumulatesAndIsLocked
```

РЕКОМЕНДАЦИЯ

Добавить функцию вывода накопленной выручки на адрес казны под отдельной ролью, с событием и с проверкой, что выводимая сумма не превышает учтённую.

Если по замыслу выручка должна возвращаться в фонд А и увеличивать доступную ликвидность, это тоже нужно выразить в коде явно, а не оставлять следствием того, что средства просто лежат на балансе. Сейчас поведение не соответствует ни одному из осмысленных вариантов: средства и не распределяются, и не работают.

В любом случае выбранную схему стоит зафиксировать в спецификации, поскольку сейчас она не описана ни там, ни в коде.

M-04

СРЕДНЯЯ

ОТКРЫТО

Минимальный размер займа не проверяется, что позволяет создавать позиции, невыгодные для ликвидации

ВЛИЯНИЕ

Среднее

ВЕРОЯТНОСТЬ

Средняя

КАТЕГОРИЯ

Учёт пула

РАСПОЛОЖЕНИЕ

`src/core/ICFT/lending/LendingPool.sol#L277-L279`

ОПИСАНИЕ

Единственная проверка размера займа это отличие от нуля.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
277 function borrow(uint256 amountICFT) external nonReentrant whenNotPaused {
278     if (amountICFT == 0) revert ZeroAmount();
```

Нижней границы в денежном выражении нет, поэтому позиция может быть открыта на одну неделимую единицу ICFT. Ограничение сверху при этом есть: проверяется и LTV, и предел утилизации.

ВЛИЯНИЕ

Ликвидация приносит ликвидатору бонус в пять процентов от покрытого долга и стоит ему газа. Отсюда размер долга, ниже которого закрывать позицию экономически бессмысленно. При цене газа в двести gwei и ETH около \$1 600 порог рентабельности составляет примерно \$1 920, при четырёхстах gwei он удваивается.

Позиции ниже этого порога не будут закрыты никем, потому что ликвидатор потеряет на газе больше, чем получит. Они останутся в системе и при неблагоприятном движении цены перейдут в непокрытый долг, описанный в H-02.

Само по себе это медленная утечка, а не разовая потеря. Но её можно ускорить намеренно: создание большого числа мелких позиций стоит атакующему только газа, а протоколу оставляет долг, который невозможно закрыть с прибылью. Дополнительной нагрузки на циклы в контракте при этом не возникает, позиции нигде не перебираются, так что отказа в обслуживании здесь нет, вред чисто экономический.

ДОКАЗАТЕЛЬСТВО (POC)

Заём на одну неделимую единицу ICFT проходит успешно и создаёт позицию с ненулевым масштабированным долгом.

```
1 forge test --mt test_P3_03_dustBorrowAccepted
```

РЕКОМЕНДАЦИЯ

Ввести минимальный размер займа в денежном выражении и проверять его в `borrow()` после конвертации суммы в USD. Величину имеет смысл сделать настраиваемой, поскольку порог рентабельности зависит от цены газа и от сети, в которой развёрнут протокол.

Отдельно стоит предусмотреть минимальный остаток позиции после частичной ликвидации, чтобы закрытие позиции не оставляло за собой остаток, который снова окажется ниже порога рентабельности.

M-05

СРЕДНЯЯ

ОТКРЫТО

Передача административной роли одношаговая, а пауза не ограничена по времени

ВЛИЯНИЕ

Среднее

ВЕРОЯТНОСТЬ

Низкая

КАТЕГОРИЯ

Права доступа и апгрейд

РАСПОЛОЖЕНИЕ

```
src/core/ICFT/lending/LendingPool.sol#L399-L405  
src/core/ICFT/lending/LendingPool.sol#L204-L211
```

ОПИСАНИЕ

Права построены на `AccessControlUpgradeable`. Роли выдаются и отзываются одним вызовом, подтверждения со стороны принимающего адреса не требуется. Вариант `AccessControlDefaultAdminRules`, который добавляет двухшаговую передачу и задержку, не используется.

Отдельный вопрос это пауза. Снять её может только держатель `PAUSER_ROLE`, автоматического снятия по истечении срока нет.

src/core/ICFT/lending/LendingPool.sol

solidity

```
399 function pause() external onlyRole(PAUSER_ROLE) {  
400     _pause();  
401 }  
402  
403 function unpause() external onlyRole(PAUSER_ROLE) {  
404     _unpause();  
405 }
```

ВЛИЯНИЕ

Ошибка в адресе при передаче `DEFAULT_ADMIN_ROLE` необратима. Если роль выдана на адрес, к которому нет доступа, и следом отозвана у прежнего держателя, управление протоколом теряется навсегда, а вместе с ним и возможность что-либо исправить.

Бессрочная пауза означает, что недоступность одного ключа замораживает протокол на неопределённый срок. С учётом M-02, где во время паузы обеспечение заперто, а проценты продолжают идти, это превращается из неудобства в прямой ущерб пользователям.

Вероятность низкая, поскольку оба сценария требуют операционной ошибки или потери ключа, а не эксплуатации кода.

РЕКОМЕНДАЦИЯ

Перейти на `AccessControlDefaultAdminRules` для роли администратора: она требует подтверждения со стороны нового держателя и вводит обязательную задержку, что делает ошибочную передачу заметной и обратной.

Для паузы предусмотреть предельный срок, по истечении которого она снимается автоматически либо требует повторного подтверждения. Это ограничивает последствия недоступности ключа и заодно дисциплинирует процесс реагирования на инцидент.

Дополнительно стоит развести держателей ролей по разным адресам. Сейчас все роли сходятся к одному ключу, поэтому его потеря означает не частичную, а полную утрату управления.

L-01

НИЗКАЯ

ОТКРЫТО

Ответ Chainlink проверяется не полностью: номер раунда игнорируется, запас по устареванию отсутствует

КАТЕГОРИЯ

Оракульный слой

РАСПОЛОЖЕНИЕ

src/core/ICFT/oracle/PriceOracle.sol#L298-L307

ОПИСАНИЕ

Чтение фида проверяет знак ответа и его возраст, но разбирает не все возвращаемые поля.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
301 (, int256 answer,, uint256 updatedAt,) = feed.latestRoundData();
302
303 if (answer <= 0 || updatedAt == 0) revert InvalidOracleAnswer();
304 if (block.timestamp > updatedAt + maxPriceAge) revert StaleOraclePrice();
```

Поле `answeredInRound` не читается и с `roundId` не сравнивается. Проверка `updatedAt == 0` закрывает случай незавершённого раунда, поэтому практический эффект меньше, чем у классического варианта этой проблемы, но формально полнота ответа не подтверждается.

Отдельно стоит отметить настройку `maxPriceAge`. В развёртывании она равна одному часу, что совпадает с периодом обновления фида ETH/USD. Запаса нет, и при штатной задержке публикации обновления цена считается устаревшей.

Проверки работоспособности секвенсора для сетей второго уровня в оракуле нет. Для текущего скоупа это не критично, но при развёртывании на L2 добавится окно, в котором цена формально свежая, а сеть только что восстановилась.

ВЛИЯНИЕ

Когда цена признана устаревшей, откатываются все операции, которым нужна оценка обеспечения, включая погашение и ликвидацию. Заёмщик в этот момент не может ни уменьшить долг, ни быть закрытым, а проценты продолжают начисляться. То есть ошибка на стороне фида превращается во временную блокировку выхода.

РЕКОМЕНДАЦИЯ

Читать `roundId` и `answeredInRound` и отклонять ответ, если раунд неактуален. Задать `maxPriceAge` с запасом относительно периода обновления фида, обычно берут полтора или два периода, чтобы штатная задержка публикации не приводила к отказу.

При переходе на сети второго уровня добавить чтение фида доступности секвенсора и период ожидания после его восстановления.

L-02

НИЗКАЯ

ОТКРЫТО

Валидация параметров риска допускает нулевой запас между целевым LTV и порогом ликвидации

КАТЕГОРИЯ

Параметры риска

РАСПОЛОЖЕНИЕ

src/core/ICFT/risk/RiskEngine.sol#L233-L245

ОПИСАНИЕ

Проверка соотношения целевого LTV и порога ликвидации нестрогая.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
241 || targetLtvBps_ > liquidationThresholdBps_
```

Значения принимаются и при равенстве. В этом случае ликвидация приводит позицию ровно к порогу, то есть запас здоровья после закрытия равен нулю.

Первоначально это поведение выглядело как петля повторных ликвидаций. Проверка показала иное: после приведения к цели позиция остаётся формально подлежащей ликвидации из-за округления

`calculateLTV` вниз, допускает ровно один дополнительный вызов, в котором изымается одна неделимая единица обеспечения из-за нижней отсечки `seizeAmount`, после чего LTV опускается ниже порога и процесс останавливается.

ВЛИЯНИЕ

Прямой ущерб ничтожен и измеряется одной неделимой единицей обеспечения. Существенно другое: при такой настройке позиция после ликвидации остаётся ровно на границе и любое минимальное движение цены снова делает её подлежащей закрытию. Пользователь платит бонус ликвидатору повторно, не получая устойчивого запаса.

РЕКОМЕНДАЦИЯ

Сделать проверку строгой и потребовать заметный зазор между целевым LTV и порогом ликвидации, порядка трёх или пяти процентных пунктов. Тогда ликвидация будет возвращать позицию в состояние, которое переживает небольшие колебания цены.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
- || targetLtvBps_ > liquidationThresholdBps_  
+ || targetLtvBps_ + MIN_LTV_GAP_BPS > liquidationThresholdBps_
```

L-03

НИЗКАЯ

ОТКРЫТО

Сборка проекта не воспроизводится: конфликт путей зависимостей и нестрогая версия компилятора

КАТЕГОРИЯ

Сборка и зависимости

РАСПОЛОЖЕНИЕ

```
remappings.txt  
foundry.toml  
package.json
```

ОПИСАНИЕ

Проект не собирается из состояния репозитория. Причин несколько и они накладываются друг на друга.

Файл `remappings.txt` перекрывает настройки из `foundry.toml` и направляет `@openzeppelin/` и `@chainlink/` в `node_modules/`, где установлены версии с другой раскладкой каталогов. В результате два импорта не разрешаются: `ReentrancyGuardUpgradeable` отсутствует по ожидаемому пути в установленной версии OpenZeppelin, а `AggregatorV3Interface` в установленной версии Chainlink лежит не в `shared/interfaces`, как указано в импорте. Каталог `lib/` содержит ещё один набор зависимостей, частично несовместимый с первым.

Отдельно сборку ломает файл `script/CheckSepoliaState.s.sol`: в нём символ эмодзи помещён в обычный строковый литерал, что для компилятора является ошибкой разбора.

Все контракты объявлены с нестрогой версией `^0.8.20`, тогда как фактически компиляция идёт версией `0.8.35`.

ВЛИЯНИЕ

Невозможность собрать проект из репозитория мешает независимой проверке и воспроизведению результатов. Нестрогая версия компилятора означает, что байткод зависит от того, какую версию подберёт инструмент сборки, поэтому проверка соответствия развёрнутого байткода исходникам усложняется.

Для аудита зависимости были довендорены вручную, а сборка скриптов исключена. Эти изменения носят временный характер и не входят в рекомендации.

РЕКОМЕНДАЦИЯ

Свести зависимости к одному источнику: либо `lib/` через git-подмодули, либо `node_modules/`, но не оба одновременно. Зафиксировать точные версии OpenZeppelin и Chainlink, соответствующие тем путям импорта, которые используются в коде. Удалить конфликтующие записи из `remappings.txt`, оставив разрешение путей в `foundry.toml`.

Зафиксировать версию компилятора точно, например `pragma solidity 0.8.35;`, и указать её же в настройках сборки. В строковый литерал с эмодзи добавить префикс `unicode`.

L-04

НИЗКАЯ

ОТКРЫТО

Оценка обеспечения делает внешний вызов на каждый актив в цикле по всему списку

КАТЕГОРИЯ

Учёт пула

РАСПОЛОЖЕНИЕ

src/core/ICFT/lending/LendingPool.sol#L685-L702
src/core/ICFT/lending/LendingPool.sol#L762-L780

ОПИСАНИЕ

Подсчёт стоимости обеспечения перебирает весь список поддерживаемых активов и для каждого ненулевого баланса обращается к `RiskEngine`, который, в свою очередь, обращается к оракулу.

src/core/ICFT/lending/LendingPool.sol

solidity

```
692 for (uint256 i = 0; i < assetsLength; ++i) {
693     address asset = supportedCollateralAssets[i];
694     uint256 balance = _getCollateralBalanceMemory(user, position, asset);
695
696     if (balance == 0) {
697         continue;
698     }
699
700     collateralValueUSD += riskEngine.getCollateralValueUSD(asset, balance);
701 }
```

Тот же шаблон используется в `_hasAnyCollateral`. Массив `supportedCollateralAssets` пополняется через `setCollateralAsset` и никогда не сокращается: отключённый актив остаётся в списке, потому что удаление из массива не предусмотрено.

ВЛИЯНИЕ

Расход газа на `borrow`, `repay`, `withdrawCollateral` и ликвидацию растёт линейно с числом когда-либо добавленных активов, независимо от того, сколько из них реально используется заёмщиком и сколько остаются включёнными. Сегодня активов немного, поэтому влияние незначительное. При дальнейшем расширении списка операции будут дорожать для всех пользователей, а при достаточно длинном списке ликвидация может стать невыгодной раньше, чем этого ожидает экономическая модель.

РЕКОМЕНДАЦИЯ

Ограничить длину списка поддерживаемых активов явной константой и проверять её при добавлении. Предусмотреть исключение актива из массива, а не только снятие флага `enabled`, чтобы отключённые активы не увеличивали стоимость каждой операции.

Как вариант, хранить для каждой позиции список фактически внесённых активов и перебирать его, а не общий реестр. Тогда стоимость операции будет зависеть от состава конкретной позиции, а не от истории настроек протокола.

L-05

НИЗКАЯ

ОТКРЫТО

В обновляемых контрактах не зарезервированы слоты хранилища

КАТЕГОРИЯ

Права доступа и апгрейд

РАСПОЛОЖЕНИЕ

src/core/ICFT/lending/LendingPool.sol#L155-L179
src/core/ICFT/oracle/PriceOracle.sol#L127-L137

ОПИСАНИЕ

Все контракты протокола обновляемые и работают за прокси. Собственное состояние размещено в последовательных слотах, начиная с нулевого. Резервного массива вида `uint256[N] __gap` нет ни в одном контракте.

Базовые контракты OpenZeppelin пятой версии используют именованные пространства хранилища, поэтому коллизии между ними и наследником не возникает. Риск ограничен собственным состоянием протокола.

Код содержит комментарии о том, что порядок полей менять нельзя, и порядок действительно соблюдается. Проект уже проходил через повторную инициализацию с добавлением реестра залоговых активов, то есть апгрейды применяются на практике.

src/core/ICFT/lending/LendingPool.sol

solidity

```
162 // Append-only storage layout:
163 // keep legacy state fields in their original order so already deployed
164 // proxies continue to read existing accounting values from the same slots.
```

ВЛИЯНИЕ

Пока новые поля добавляются строго в конец, раскладка остаётся совместимой. Дисциплина при этом держится на комментарии и внимательности разработчика, а не на технической гарантии. Вставка поля в середину или введение промежуточного контракта в цепочку наследования сдвинет слоты и приведёт к тому, что уже развёрнутый прокси прочитает по старым адресам чужие значения. Для контракта, хранящего обеспечение и учёт долга, это означает порчу данных.

РЕКОМЕНДАЦИЯ

Добавить в конец каждого обновляемого контракта резервный массив и уменьшать его размер при добавлении новых полей. Как альтернатива, перевести собственное состояние на именованные пространства хранилища по той же схеме, что используют базовые контракты OpenZeppelin.

Независимо от выбранного варианта, полезно снимать раскладку хранилища при сборке и сравнивать её с предыдущей версией автоматически. Это превращает договорённость о порядке полей в проверку, которая не зависит от человеческого внимания.

L-06

НИЗКАЯ

ОТКРЫТО

Обеспечение засчитывается по запрошенной сумме, а не по фактически полученной

КАТЕГОРИЯ

Учёт пула

РАСПОЛОЖЕНИЕ

src/core/ICFT/lending/LendingPool.sol#L250-L266

ОПИСАНИЕ

При внесении обеспечения в токенах баланс позиции увеличивается на запрошенную сумму сразу после перевода, без проверки того, сколько токенов фактически поступило на контракт.

src/core/ICFT/lending/LendingPool.sol

solidity

```
260 IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
261 ERC20CollateralBalances[msg.sender][asset] += amount;
```

Для обычных токенов ERC-20 это корректно. Для токенов, удерживающих комиссию при переводе, а также для токенов с изменяемым балансом, учтённая сумма окажется больше фактически полученной.

ВЛИЯНИЕ

Разница между учтённым и реальным обеспечением работает в пользу пользователя и против протокола: заём выдаётся под обеспечение, которого на контракте нет. При выводе или ликвидации перевод будет исходить из завышенного учтённого баланса, и нехватка проявится на последнем пользователе, чьи средства не удастся выдать.

Сегодня риск гипотетический. Список активов формируется администратором, в нём только ETH и обычные токены, а спецификация относит нестандартное поведение токенов к допущениям модели доверия. Находка зафиксирована как предупреждение на случай расширения списка.

РЕКОМЕНДАЦИЯ

Считать зачисляемую сумму как разность балансов контракта до и после перевода. Это делает учёт корректным независимо от поведения токена и обходится в одно дополнительное чтение баланса.

src/core/ICFT/lending/LendingPool.sol

solidity

```
+ uint256 balanceBefore = IERC20(asset).balanceOf(address(this));
  IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
- ERC20CollateralBalances[msg.sender][asset] += amount;
+ uint256 received = IERC20(asset).balanceOf(address(this)) - balanceBefore;
+ ERC20CollateralBalances[msg.sender][asset] += received;
```

Если поддержка таких токенов не планируется, ограничение следует зафиксировать в документации и проверять при добавлении актива в реестр.

L-07

НИЗКАЯ

ОТКРЫТО

Ступенчатая кривая ставки даёт скачок на границе участка и не ограничена сверху

КАТЕГОРИЯ

Модель ставки

РАСПОЛОЖЕНИЕ

src/core/ICFT/lending/InterestRateModel.sol#L136-L143
src/core/ICFT/lending/InterestRateModel.sol#L162-L179

ОПИСАНИЕ

Ставка возвращается как одно из четырёх фиксированных значений в зависимости от участка утилизации, без интерполяции внутри участка.

src/core/ICFT/lending/InterestRateModel.sol

solidity

```
136 function getBorrowRateBps(uint256 utilizationBps) external view returns (uint256) {
137     RateConfig memory config = rateConfig;
138
139     if (utilizationBps < config.kink1Bps) return config.rate1Bps;
140     if (utilizationBps < config.kink2Bps) return config.rate2Bps;
141     if (utilizationBps < config.kink3Bps) return config.rate3Bps;
142     return config.rate4Bps;
143 }
```

Значения кривой совпадают со спецификацией MVP, поэтому вопрос не к самим числам, а к форме: на границе участка ставка меняется скачком, например с восьми процентов до пятнадцати при переходе через отметку восемьдесят процентов утилизации.

Валидация конфигурации проверяет порядок точек излома и неубывание ставок, но не задаёт верхнего предела. Значение `rate4Bps` может быть произвольно большим.

ВЛИЯНИЕ

Скачок на границе создаёт заметную разницу в стоимости займа при минимальном изменении утилизации, что даёт повод подгадывать момент операции. Влияние ограничено тем, что начисление привязано к моменту изменения состояния и внутри одного блока не накапливается, поэтому выгода от такой подгонки мала.

Отсутствие верхнего предела ставки означает, что изменение конфигурации может резко увеличить стоимость уже открытых займов. Это относится к полномочиям роли и описано в модели доверия, но ограничение на уровне кода было бы уместнее.

РЕКОМЕНДАЦИЯ

Рассмотреть переход к кусочно-линейной кривой с интерполяцией внутри участка. Такая форма принята в большинстве кредитных протоколов именно потому, что убирает разрыв на границе и делает стоимость займа предсказуемой.

Задать в валидации разумный верхний предел годовой ставки, чтобы изменение конфигурации не могло сделать обслуживание долга произвольно дорогим.

I-01

ИНФОРМАЦИОННАЯ

ОТКРЫТО

Нестрогая версия компилятора и неполная документация публичного интерфейса

КАТЕГОРИЯ

Качество кода

РАСПОЛОЖЕНИЕ

`src/core/**/*.sol`

ОПИСАНИЕ

Все шестнадцать файлов объявлены с нестрогой версией `pragma solidity ^0.8.20`, тогда как фактическая сборка идёт компилятором `0.8.35`. Версия выше `0.8.20`, поэтому известных проблем ранних версий здесь нет, но зависимость байткода от выбора инструмента остаётся.

Проверка стилем дала двести пятьдесят замечаний к `NatSpec`: у части публичных и внешних функций отсутствуют описания, теги параметров и возвращаемых значений либо не совпадают с сигнатурой. Тридцать шесть замечаний относятся к событиям, у которых ключевые поля не помечены как индексруемые.

Отдельно инструмент отметил размер `LendingPool`: восемнадцать переменных состояния при рекомендуемом пределе в пятнадцать и одна функция длиной пятьдесят пять строк.

ВЛИЯНИЕ

Прямого влияния на безопасность нет. Неполный `NatSpec` затрудняет независимую проверку и работу интеграторов, а отсутствие индексруемых полей в событиях усложняет офлайн-наблюдение за протоколом, что напрямую касается реагирования на инциденты.

РЕКОМЕНДАЦИЯ

Зафиксировать версию компилятора точно и привести её в соответствие с настройками сборки. Дополнить `NatSpec` для всего публичного интерфейса, в первую очередь для функций, работающих со средствами и с параметрами риска. Пометить индексруемыми адреса и идентификаторы в событиях, по которым строится мониторинг.

Крупные функции расчёта имеет смысл разделить, это упростит и чтение, и последующие проверки.

I-02

ИНФОРМАЦИОННАЯ

ОТКРЫТО

Внешний вайтпейпер описывает протокол иначе, чем спецификация MVP и реализация

КАТЕГОРИЯ

Документация

РАСПОЛОЖЕНИЕ

docs/MVP_SPEC.md

ОПИСАНИЕ

В работе использовались два документа: внешний вайтпейпер протокола и спецификация `docs/MVP_SPEC.md` из репозитория. По ряду пунктов они расходятся, и код в каждом таком случае следует спецификации, а не вайтпейперу.

Ниже перечислено то, что при сверке только с вайтпейпером выглядело бы как дефект реализации, но таковым не является.

Ликвидация. Вайтпейпер описывает её как открытую для любого участника. В коде оба входа закрыты ролями. Спецификация прямо допускает такую схему: «Restricted liquidation executor or bot is acceptable for MVP» и отдельно оговаривает, что полноценный неразрешительный механизм на MVP не требуется.

Кривая ставки. Вайтпейпер приводит участки `5 / 7 / 10 / 15 / 25` процентов. Спецификация задаёт `5 / 8 / 15 / 20`, и реализация совпадает с ней точно, включая точки излома.

Метрика утилизации. Спецификация предписывает считать её как отношение выданных ICFT к объёму фонда A, что код и делает. Замечание к числителю вынесено отдельно в M-01 и касается способа его ведения, а не выбора метрики.

Ручная цена ICFT. Спецификация описывает её как решение для MVP с последующим переходом на рыночный источник. Ограничители при этом не отменяются, поэтому H-01 остаётся в силе.

Собственный обменник, маршрутизация обмена, механика управления и финальная схема прокси перечислены в спецификации в разделе явно исключённого из объёма.

Резервный фонд B, вестинг и минимальный размер займа в спецификации не упоминаются вовсе. Это пункты вайтпейпера, и на текущем этапе они вне объёма. Оговорюсь, что вопрос покрытия убытков от неплатёжеспособности при этом остаётся открытым независимо от судьбы фонда B, он вынесен в H-02, а минимальный размер займа имеет самостоятельное экономическое обоснование и вынесен в M-04.

ВЛИЯНИЕ

Расхождение документов само по себе не влияет на безопасность контрактов, но влияет на ожидания. Читатель вайтпейпера рассчитывает на открытую ликвидацию, вестинг и резервный фонд, тогда как развёрнутая система работает иначе. Для аудита это создаёт риск неверной классификации находок, для пользователя это риск неверных решений.

РЕКОМЕНДАЦИЯ

Привести внешний вайтпейпер в соответствие с фактическим объёмом MVP либо явно разделить в нём целевую архитектуру и текущий этап, указав, что реализовано сейчас, а что запланировано.

До запуска в основной сети следует вернуться к пунктам, отложенным за рамки MVP, и принять по каждому решение: реализовать или зафиксировать отказ. Это относится прежде всего к открытой ликвидации, вестингу и механизму покрытия убытков.

I-03

ИНФОРМАЦИОННАЯ

ОТКРЫТО

Проценты начисляются сложным способом, а режим начисления в спецификации не зафиксирован

КАТЕГОРИЯ

Модель ставки

РАСПОЛОЖЕНИЕ

`src/core/ICFT/lending/LendingPool.sol#L546-L575`

ОПИСАНИЕ

Начисление идёт через глобальный индекс. Приращение считается от текущего значения индекса, а не от исходного.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
559 uint256 elapsed = block.timestamp - previousAccrualTime;
560 uint256 aprBps = interestRateModel.getBorrowRateBps(_calculateUtilization(totalBorrowedICFT));
561 uint256 interestFactor = (previousBorrowIndex * aprBps * elapsed) / (BPS * YEAR);
```

Поскольку множителем выступает индекс на момент начисления, каждое последующее начисление считается от уже увеличенной базы. Это сложный процент с дискретной капитализацией, и период капитализации определяется не расписанием, а частотой обращений к протоколу: чем чаще пользователи совершают операции, тем чаще происходит начисление и тем выше эффективная ставка за год.

Отдельно отмечу, что механика переоценки прошедших периодов здесь реализована корректно. Начисление вызывается перед каждым изменением состояния, а утилизация между двумя начислениями не меняется, поэтому истёкший период всегда тарифицируется по действовавшей на нём ставке.

Спецификация MVP говорит только о том, что проценты начисляются в USD и считаются от основного долга в USD. Про простой или сложный процент в ней ничего нет.

ВЛИЯНИЕ

Заёмщик платит несколько больше, чем следует из номинальной годовой ставки, и величина этой разницы зависит от активности других пользователей, то есть не определяется заранее. При ставке в двадцать процентов и частых обращениях эффективная годовая ставка приближается к двадцати двум процентам.

Ошибкой это считать нельзя, поскольку никакому зафиксированному требованию поведение не противоречит. Находка внесена, чтобы зафиксировать расхождение между реализацией и распространённым ожиданием простого процента.

РЕКОМЕНДАЦИЯ

Зафиксировать выбранный режим начисления в спецификации явно. Если задумывался сложный процент, стоит указать это и описывать ставку как эффективную годовую, чтобы пользователь видел реальную стоимость займа.

Если задумывался простой процент, приращение следует считать от исходного значения индекса, а не от текущего, и отразить это в модели.

ПРИЛОЖЕНИЕ А

Инварианты и тесты свойств

Свойства ниже реализованы как инварианты Foundry с состоянием в ходе аудита и переданы команде для включения в непрерывную интеграцию. Обработчик двигает в том числе цены, что отличает набор от существовавшего в репозитории.

ID	ИНВАРИАНТ	МЕТОД	РЕЗУЛЬТАТ
INV-01	Сумма долгов всех позиций совпадает с совокупным учтённым долгом с точностью до одной единицы на позицию.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-02	Баланс ETH на контракте не меньше суммы обеспечения всех позиций.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-03	Эмиссия ICFT постоянна и равна одному миллиарду при любой последовательности операций.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-04	Действие пользователя не может оставить его позицию с LTV выше максимального.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-05	Частичная ликвидация приводит LTV к целевому значению и не превышает необходимого размера.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-06	Начисленные проценты монотонно не убывают, долг не уменьшается без погашения.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-07	Баланс ICFT на контракте покрывает учтённую выручку протокола.	Фаззинг с состоянием	ВЫПОЛНЯЕТСЯ
INV-08	Ликвидность фонда А вместе с выданными токенами равна выделенному объёму фонда.	Фаззинг с состоянием	НАРУШЕН → M-01
INV-09	Позиция с обеспечением ниже долга закрывается полностью, непокрытого остатка не возникает.	Фаззинг с состоянием	НАРУШЕН → H-02
INV-10	Формулы LTV, начисления процентов и частичной ликвидации совпадают с эталонной моделью на случайных входах.	Дифференциальное тестирование	ВЫПОЛНЯЕТСЯ

ПРИЛОЖЕНИЕ В

Разбор автоматического анализа

Каждое срабатывание статических анализаторов разобрано вручную. Ни один автоматический результат не стал находкой сам по себе, два легли в основу ручного анализа.

ИНСТРУМЕНТ	ДЕТЕКТОР	СРАБАТЫВАНИЙ	РАЗБОР
Slither	arbitrary-send-eth	1	Ложное срабатывание по эксплуатируемости: функция закрыта ролью и проверяет адрес, сумму и результат. Полномочие описано в модели доверия.
Slither	weak-prng	1	Ложное срабатывание: остаток от деления служит округлению вверх, а не источником случайности.
Slither	incorrect-equality	17	Строгие сравнения с внутренне вычисленными величинами, а не с наблюдаемым балансом. Влияния нет.
Slither	divide-before-multiply	1	Значения в масштабе 1e18, деление округляется вверх в пользу протокола. Потеря точности на уровне единицы.
Slither	unused-return	1	Подтверждено, легло в основу L-01 : часть полей ответа Chainlink не проверяется.
Slither	calls-loop	8	Подтверждено, вынесено в L-04 : внешние вызовы в цикле по списку залоговых активов.
Slither	timestamp	19	Допустимо: время используется для расчёта истёкшего периода и проверки устаревания цены.
solhint	use-natspec	250	Подтверждено, вынесено в I-01 : документация публичного интерфейса неполная.
solhint	gas-indexed-events	36	Подтверждено: ключевые поля событий не индексированы, что усложняет мониторинг.

ПРИЛОЖЕНИЕ С

Ограничение ответственности

Настоящий отчёт не является и не должен рассматриваться как одобрение или неодобрение какого-либо проекта или команды. Он не содержит гарантий отсутствия ошибок в проанализированной технологии и не даёт оценки её владельцам, бизнес-модели, деловому или правовому соответствию.

Аудит выполнен в отношении исходного кода на коммите, указанном в разделе «Объём аудита». Любые изменения кода после этого коммита, а также любой деплой, не совпадающий с ним побайтно, находятся за пределами предоставленных заверений. Пользователям следует убедиться, что развёрнутый байткод соответствует проверенному исходному коду.

Аудит безопасности это ограниченная по времени работа, выполняемая с максимальной тщательностью. Он снижает, но не устраняет риск. Аудитор рекомендует несколько независимых проверок, публичную программу вознаграждения за найденные уязвимости и постоянный мониторинг для любой системы, хранящей средства пользователей.

Заказчик вправе публиковать отчёт только целиком и без изменений.



WhyUSayMyName

Независимый аудит безопасности смарт-контрактов

github.com/WhyUSayMyName

x.com/WhyUSayMyName